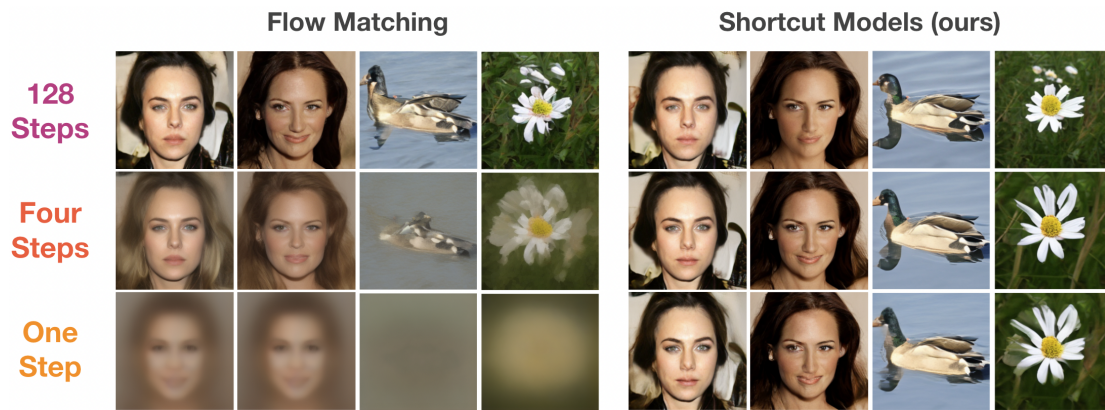


Tài liệu repo shortcut-models

Shortcut Models trên JAX/Flax

Phiên bản PDF tổng hợp từ README và hướng dẫn nội bộ của repo.



Repo này hiện thực Shortcut Models cùng nhiều baseline liên quan trên JAX/Flax. Tài liệu tập trung vào bốn phần: mục tiêu kỹ thuật của repo, cấu trúc mã nguồn, luồng train và inference, cùng các lệnh vận hành thực tế để khởi chạy, đánh giá và tránh các lỗi cấu hình thường gặp.

Mục lục

1	Tổng quan	2
1.1	Mục tiêu của repo	2
1.2	Các mode huấn luyện	2
1.3	Phạm vi tài liệu PDF này	2
2	Cấu trúc repo và luồng thực thi	3
2.1	Các file chính	3
2.2	Luồng train	3
2.3	Luồng inference	3
2.4	Điểm khác giữa tài liệu cũ và code hiện tại	4
3	Thiết lập và cách chạy	4
3.1	Thiết lập môi trường	4
3.2	Dataset đang hỗ trợ	4
3.3	Các cờ quan trọng	5
3.4	Ví dụ lệnh train	5
3.5	Ví dụ lệnh inference	7
4	Checkpoint, đánh giá và các lưu ý vận hành	8
4.1	Checkpoint và logging	8
4.2	Sanity check mục tiêu	8
4.3	Những điểm nên chú ý trước khi chạy thật	8
4.4	Thứ tự đọc code đề xuất	8
4.5	Kết luận	9

1 Tổng quan

Repo `shortcut-models` hiện thực bài toán sinh ảnh bằng *Shortcut Models* trên nền JAX/Flax. Ý tưởng chính là huấn luyện một mô hình DiT duy nhất để vừa học mục tiêu flow matching tiêu chuẩn, vừa học thêm các shortcut target ở nhiều độ dài bước khác nhau. Nhờ đó, cùng một checkpoint có thể được dùng cho nhiều ngân sách lấy mẫu, từ nhiều bước xuống một bước.

1.1 Mục tiêu của repo

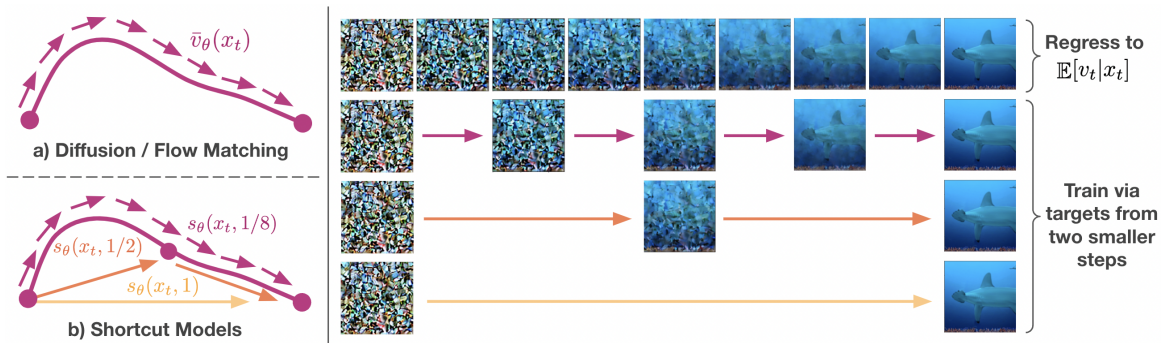
Ba mục tiêu vận hành chính của repo là:

- huấn luyện Shortcut Models theo bài báo gốc;
- giữ các baseline để so sánh, gồm naive, sit, progressive, consistency, consistency-distillation và livereflow;
- cung cấp cùng một entripoint cho train, đánh giá định kỳ và sampling offline.

1.2 Các mode huấn luyện

Repo hiện hỗ trợ các giá trị `-model.train_type` sau:

- `shortcut`: mode chính của bài báo. Batch được trộn giữa flow target và bootstrap shortcut target.
- `naive`: flow matching cơ bản, không dùng shortcut target.
- `sit`: baseline Scalable Interpolant Transformers dùng cùng backbone DiT nhưng đổi sang transport continuous-time với path, prediction target và sampler cấu hình được.
- `progressive`: dùng teacher state để rút ngắn dần bước theo tiến độ train.
- `consistency`: consistency training dựa trên EMA.
- `consistency-distillation`: teacher-student consistency distillation.
- `livereflow`: sinh một phần target kiểu reflow trực tiếp từ rollout của model hiện tại.



Hình 1: Hình minh họa trực quan cho Shortcut Models trong tài liệu gốc.

1.3 Phạm vi tài liệu PDF này

Tài liệu tập trung vào các nội dung cần cho việc vận hành repo:

- file nào là entripoint chính và file nào chứa target/model;

- luồng dữ liệu từ dataset sang latent, target và update;
- lệnh thiết lập môi trường, train và inference phù hợp với code hiện tại;
- các điểm dễ gây nhầm lẫn giữa tài liệu cũ và hành vi thực tế của nguồn.

2 Cấu trúc repo và luồng thực thi

2.1 Các file chính

File/thư mục	Vai trò
<code>train.py</code>	Entry point chính. Dùng để parse flags, tạo dataset, khởi tạo model, chạy train loop, eval định kỳ và chuyển sang inference path khi <code>-mode</code> khác train.
<code>model.py</code>	Định nghĩa backbone DiT với conditioning theo thời gian t , độ dài bước dt và nhãn lớp.
<code>targets_shortcut.py</code> <code>baselines/</code>	Sinh target cho thuật toán shortcut. Đây là phần cốt lõi của repo. Chứa các target builder cho naive, sit, progressive, consistency, consistency-distillation và livereflood.
<code>utils/sit/_transport.py</code>	Gom toán transport cho SiT: path linear/gvp/vp, target velocity/score/noise, weighting và sampler ODE/SDE fixed-step.
<code>helper_eval.py</code>	Visualization, reconstruction, sample grids, activation logging và tính FID trong lúc train.
<code>helper_inference.py</code>	Sampling/FID path khi không train.
<code>utils/datasets.py</code>	Pipeline TFDS cho các dataset đang hỗ trợ.
<code>utils/stable_vae.py</code>	Wrapper encode/decode latent qua Stable VAE.
<code>utils/sharding.py</code>	Thiết lập sharding dp hoặc fsdp.
<code>utils/checkpoint.py</code>	Lưu và đọc checkpoint dạng pickle đơn giản.
<code>utils/train_state.py</code>	Train state của Flax cùng EMA parameters.

2.2 Luồng train

Trong mode train, file `train.py` đi theo chuỗi bước sau:

1. parse toàn bộ flags top-level và config con trong `model.*` và `wandb.*`;
2. tạo dataset train/validation qua TFDS;
3. nếu `-model.use_stable_vae=1`, encode ảnh đầu vào sang latent trước khi đưa vào DiT;
4. khởi tạo model, optimizer adamw và train state có EMA;
5. chọn hàm sinh target dựa trên `-model.train_type`;
6. lặp qua từng batch để tính target, loss, gradient và cập nhật trọng số;
7. định kỳ log W&B, chạy `helper_eval.py` và lưu checkpoint.

2.3 Luồng inference

Nếu `-mode` khác train, `train.py` sẽ chuyển sang `helper_inference.py`. Luồng này sẽ:

- load checkpoint nếu có `-load_dir`;

- tạo noise đầu vào;
- chạy Euler sampling cho các mode legacy, hoặc transport ODE/SDE fixed-step cho sit;
- nếu có `-fid_stats`, tính FID với Inception activations;
- lưu `x\_render.npy` khi helper thực sự thu thập ảnh render vào `save\_dir`.

2.4 Điểm khác giữa tài liệu cũ và code hiện tại

Có một vài điểm cần lưu ý khi đọc tài liệu cũ của repo:

- ví dụ cũ dùng dataset name `celeb_a_hq`, nhưng loader hiện tại nhận `celebahq256`;
- inference path phù hợp nhất khi có `-fid_stats`;
- nhánh `-mode interpolate` hiện vẫn chứa `breakpoint()`, nên chưa phải đường chạy ổn định để dùng ngay.

3 Thiết lập và cách chạy

3.1 Thiết lập môi trường

Repo hiện có hai cách cài đặt. Cách khuyến nghị là dùng `uv`, vì repo đã có sẵn `uv.lock`.

Listing 1: Thiết lập bằng `uv`

```
uv sync
```

Đường cài đặt legacy vẫn còn trong repo:

Listing 2: Thiết lập bằng `conda` + `pip`

```
conda env create -f environment.yml
conda activate project-brc
pip install -r requirements.txt
```

Một vài lưu ý thực tế:

- `pyproject.toml` đang pin Python 3.11.6;
- `environment.yml` lại dùng Python 3.10;
- `utils/stable_vae.py` sẽ tải VAE từ Hugging Face ở lần chạy đầu tiên;
- `utils/fid.py` sẽ tải trọng số Inception vào thư mục `data/` khi cần tính FID.

3.2 Dataset đang hỗ trợ

dataset_name	Nguồn	Ghi chú
imagenet256	TFDS imagenet2012	crop vuông rồi resize về 256×256 .
celebahq256	TFDS celebahq256	thường cần builder TFDS riêng như tài liệu gốc từng nhắc tới.
lsunchurch	TFDS lsunc	dùng split <code>church-train</code> và <code>church-test</code> .

3.3 Các cờ quan trọng

Top-level flags

Cờ	Ý nghĩa
-dataset_name	Chọn dataset cho pipeline TFDS.
-fid_stats	File .npz chứa μ và σ để tính FID.
-load_dir	File checkpoint để load cho resume hoặc inference.
-save_dir	Prefix/path đầu ra cho checkpoint hoặc artefact inference.
-mode	train theo mặc định; giá trị khác sẽ đi vào inference path.
-batch_size	Global batch size.
-max_steps	Số bước train.
-debug_overfit	Giảm dataset xuống một tập nhỏ lặp lại để debug.

Model flags hay dùng

Cờ	Ý nghĩa
-model.hidden_size	Chiều ẩn của backbone DiT.
-model.patch_size	Kích thước patch embedding.
-model.depth	Số block Transformer.
-model.num_heads	Số attention heads.
-model.mlp_ratio	Tỉ lệ mở rộng của MLP block.
-model.cfg_scale	CFG scale của model conditional.
-model.class_dropout_prob	Xác suất dropout nhãn để phục vụ classifier-free guidance.
-model.num_classes	Số lớp; với bài toán unconditional thường đặt 1.
-model.denoise_timesteps	Flow horizon chuẩn trong quá trình train.
-model.train_type	Chọn thuật toán train.
-model.sharding	dp hoặc fsdp.
-model.use_stable_vae	Encode ảnh sang latent trước khi train/sample.
-model.bootstrap_cfg	Bật CFG trong bootstrap target của shortcut/progressive.
-model.transport_path_type	Với sit: chọn interpolant path linear, gvp, hoặc vp.
-model.transport_prediction	Với sit: chọn target velocity, score, hoặc noise.
-model.transport_loss_weight	Với sit: chọn weighting none, velocity, hoặc likelihood.
-model.transport_train_eps	Với sit: override epsilon train; bỏ qua để dùng mặc định theo path/prediction.
-model.transport_sample_eps	Với sit: override epsilon sample; bỏ qua để dùng mặc định theo path/prediction.
-inference_transport	Với sit: chọn sampler ode hoặc sde.
-inference_sampling_method	Với sit: chọn solver euler hoặc heun.
-inference_diffusion_form	Với sit và sde: chọn dạng diffusion coefficient.
-inference_diffusion_norm	Với sit và sde: hệ số nhân diffusion.
-inference_last_step	Với sit và sde: hiệu chỉnh cuối none/mean/tweedie/euler.
-inference_last_step_size	Với sit và sde: độ lớn hiệu chỉnh cuối.

Mặc định epsilon của SiT đang bám logic official: linear/gvp + velocity dùng 0, linear/gvp + score/noise dùng 10^{-3} , và vp dùng $\text{train_}\epsilon=1e-5$ cùng $\text{sample_}\epsilon=1e-3$.

3.4 Ví dụ lệnh train

Listing 3: Train shortcut trên CelebA-HQ

```
uv run train.py \
```

```

--dataset_name celebahq256 \
--fid_stats data/celeba256_fidstats_ours.npz \
--save_dir checkpoints/celebahq256-shortcut/ \
--batch_size 64 \
--max_steps 410000 \
--model.hidden_size 768 \
--model.patch_size 2 \
--model.depth 12 \
--model.num_heads 12 \
--model.mlp_ratio 4 \
--model.cfg_scale 0 \
--model.class_dropout_prob 1 \
--model.num_classes 1 \
--model.train_type shortcut

```

Listing 4: Train shortcut trên ImageNet-256

```

uv run train.py \
--dataset_name imagenet256 \
--fid_stats data/imagenet256_fidstats_ours.npz \
--save_dir checkpoints/imagenet256-shortcut-b/ \
--batch_size 256 \
--max_steps 810000 \
--model.hidden_size 768 \
--model.patch_size 2 \
--model.depth 12 \
--model.num_heads 12 \
--model.mlp_ratio 4 \
--model.cfg_scale 1.5 \
--model.class_dropout_prob 0.1 \
--model.bootstrap_cfg 1 \
--model.train_type shortcut

```

Listing 5: Train shortcut trên ImageNet-256 với DiT-XL

```

uv run train.py \
--dataset_name imagenet256 \
--fid_stats data/imagenet256_fidstats_ours.npz \
--save_dir checkpoints/imagenet256-shortcut-xl/ \
--batch_size 256 \
--max_steps 810000 \
--model.hidden_size 1152 \
--model.patch_size 2 \
--model.depth 28 \
--model.num_heads 16 \
--model.mlp_ratio 4 \
--model.cfg_scale 1.5 \
--model.class_dropout_prob 0.1 \
--model.bootstrap_cfg 1 \
--model.train_type shortcut

```

Listing 6: Train baseline SiT trên ImageNet-256

```

uv run train.py \
--dataset_name imagenet256 \
--fid_stats data/imagenet256_fidstats_ours.npz \
--save_dir checkpoints/imagenet256-sit-b/ \
--batch_size 256 \
--max_steps 810000 \
--model.hidden_size 768 \

```

```

--model.patch_size 2 \
--model.depth 12 \
--model.num_heads 12 \
--model.mlp_ratio 4 \
--model.cfg_scale 1.5 \
--model.class_dropout_prob 0.1 \
--model.train_type sit \
--model.transport_path_type linear \
--model.transport_prediction velocity \
--model.transport_loss_weight none

```

3.5 Ví dụ lệnh inference

Listing 7: Sampling/FID từ checkpoint đã train

```

uv run train.py \
  --mode inference \
  --dataset_name imagenet256 \
  --fid_stats data/imagenet256_fidstats_ours.npz \
  --load_dir checkpoints/imagenet256-shortcut-b/810001 \
  --save_dir outputs/imagenet256-shortcut-b-4step \
  --batch_size 256 \
  --model.hidden_size 768 \
  --model.patch_size 2 \
  --model.depth 12 \
  --model.num_heads 12 \
  --model.mlp_ratio 4 \
  --model.cfg_scale 1.5 \
  --model.class_dropout_prob 0.1 \
  --model.bootstrap_cfg 1 \
  --model.train_type shortcut \
  --inference_timesteps 4 \
  --inference_generations 4096 \
  --inference_cfg_scale 1.5

```

Các biến thể thường dùng:

- đặt `-model.train_type naive` để train flow matching cơ bản;
- đặt `-model.train_type sit` để train baseline SiT cùng backbone DiT;
- đặt `-model.sharding fsdp` khi chạy trên nhiều thiết bị;
- đặt `-wandb.offline true` nếu chỉ muốn log cục bộ;
- đặt `-debug_overfit 1` nếu muốn kiểm tra vòng train trên dữ liệu lặp nhỏ.

Listing 8: Inference/FID cho checkpoint SiT với ODE + Heun

```

uv run train.py \
  --mode inference \
  --dataset_name imagenet256 \
  --fid_stats data/imagenet256_fidstats_ours.npz \
  --load_dir checkpoints/imagenet256-sit-b/810001 \
  --save_dir outputs/imagenet256-sit-b-ode \
  --batch_size 256 \
  --model.hidden_size 768 \
  --model.patch_size 2 \
  --model.depth 12 \

```

```

--model.num_heads 12 \
--model.mlp_ratio 4 \
--model.cfg_scale 1.5 \
--model.class_dropout_prob 0.1 \
--model.train_type sit \
--model.transport_path_type linear \
--model.transport_prediction velocity \
--model.transport_loss_weight none \
--inference_transport ode \
--inference_sampling_method heun \
--inference_timesteps 32 \
--inference_generations 4096 \
--inference_cfg_scale 1.5

```

4 Checkpoint, đánh giá và các lưu ý vận hành

4.1 Checkpoint và logging

- Repo lưu checkpoint qua `utils/checkpoint.py`.
- Trong train loop, code hiện tại nối trực tiếp `save_dir` với step number. Vì vậy nên truyền `save_dir` theo dạng có dấu gạch chéo ở cuối, ví dụ `checkpoints/run_a/`.
- Chỉ process có chỉ số 0 khởi tạo W&B khi `-mode=train`.

4.2 Sanity check mục tiêu

Các con số dưới đây là mốc tham chiếu gần đúng từ tài liệu repo để đối chiếu chất lượng train:

Model	128-step	4-step	1-step
CelebA-HQ (DiT-B)	6.9	13.8	20.5
ImageNet-256 (DiT-B)	15.5	28.3	40.3
ImageNet-256 (DiT-XL)	3.8	7.8	10.6

4.3 Những điểm nên chú ý trước khi chạy thật

1. Khi load checkpoint, cần truyền lại đúng kiến trúc model đã train, gồm `hidden_size`, `patch_size`, `depth`, `num_heads` và `mlp_ratio`.
2. Với checkpoint `sit`, cần truyền lại thêm đúng `transport\_path\_type`, `transport\_prediction`, `transport\_loss\_weight` và các cờ sampler inference nếu muốn tái lập kết quả.
3. `celebahq256` khác với `celeb_a_hq`. Dùng sai tên dataset sẽ fail ngay ở loader.
4. Inference path phù hợp nhất khi đã chuẩn bị `-fid_stats`.
5. `helper_inference.py` chỉ lưu `x\_render.npy` trong các nhánh mà helper thực sự tích lũy ảnh render.
6. `utils/stable_vae.py` và `utils/fid.py` đều có bước tải model ngoài ở lần chạy đầu tiên.

4.4 Thứ tự đọc code đề xuất

Nếu muốn hiểu repo nhanh mà vẫn nắm đúng luồng, nên đọc theo thứ tự:

1. `train.py`;

2. `targets_shortcut.py`;
3. `utils/sit\_transport.py`;
4. `model.py`;
5. `helper_eval.py`;
6. `helper_inference.py`;
7. `utils/datasets.py`.

4.5 Kết luận

PDF này được viết để thay cho phần tài liệu vận hành đang rải rác ở README và guide Markdown. Nếu repo tiếp tục thay đổi về câu lệnh, dataset hoặc train mode, nên cập nhật cả ba nơi đồng thời: README.md, docs/repo-guide.md và pdf/main.tex cùng các section liên quan.